Geavanceerde Computerarchitecturen - Flip Array

Cas Truyers en Niels Verhoeve

14 februari 2026

1 Introduction

Om kennis te maken met CUDA en de effecten van parallel processing, hebben we een programma geschreven dat een array omdraait. Er is timing-logica toegevoegd om de effecten van keuzes zoals de arraygrootte en het aantal blocks te kunnen kwantificeren. De volledige code is te vinden op: https://github.com/Niels-V-KU/Lab_Geavanceerde_Computerarchitectuur/tree/main

2 GPU Hardware

Om een idee te krijgen van de hardware die we gebruiken in het labo, hebben we het commando `nvidia-smi` gebruikt. Deze beschikt over de volgende specificaties:

- **Model:** NVIDIA T400 4GB
- **CUDA cores:** 384
- **Streaming Multiprocessors (SM):** 6

Echter hebben we voor de timing resultaten, verder in het labo, dit thuis op volgende hardware uitgevoerd:

- **Model:** NVIDIA T1200 laptop gpu
- **CUDA cores:** 1024
- **Streaming Multiprocessors (SM):** 16

3 CUDA

3.1 Code

Om de array om te draaien, mapt elke thread één waarde van de input-array naar een nieuwe array op een positie die de volgorde omdraait. Dit proces is te zien in de kernel-functie `vecReverse`.

```
__global__ void vecReverse(int* rVec, int* vec, int sizeVec) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < sizeVec) rVec[i] = vec[sizeVec - 1 - i];
}
```

3.2 Timing

3.2.1 With cudaMemcpy

In eerste instantie hebben we de timing gemeten inclusief de overhead van de dataoverdracht: het inladen van de data naar het GPU-geheugen (device), de `vecReverse` operatie zelf, en het terug overdragen van de resultaten naar de host.

```
    cudaEvent_t start_cuda, stop_cuda;
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);
    cudaEventRecord(start_cuda);

    // Allocate device memory
    cudaMalloc((void**)&d_vec, sizeVec * sizeof(int));
    cudaMalloc((void**)&d_rVec, sizeVec * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_vec, vec, sizeVec * sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel (n blocks of N threads)
    vecReverse<<<nrBlocks, nrThreads>>>(d_rVec, d_vec, sizeVec);

    // Copy result back to host
    cudaMemcpy(rVec, d_rVec, sizeVec * sizeof(int), cudaMemcpyDeviceToHost);

    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda);
    cudaEventSynchronize(stop_cuda);
    float ms;
    cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
```

De resultaten die we bij verschillende array sizes en blocks bekomen zijn te zien in tabel 1.

Array size = Threads	Blocks	Times (ms)	avg time (ms)
10	6	0.764128, 0.627520, 0.666176, 0.729376, 0.693568	0.696154
100	6	0.688480, 0.785472, 0.706816, 0.653152, 0.633696	0.693523
1000	6	0.651968, 0.621600, 0.659456, 0.581920, 0.694112	0.641811
10	1	0.658752, 0.708480, 0.637152, 0.647648, 0.713024	0.673011
100	1	0.711680, 0.725696, 0.730208, 0.724480, 0.931136	0.764640
1000	1	0.764096, 0.624224, 0.776832, 0.674176, 0.743136	0.716493

Tabel 1: Timing resultaten met dataoverdracht

3.2.2 Without cudaMemcpy

Daarna hebben we de meting aangepast om enkel de executietijd van de `vecReverse` operatie op de GPU te meten, zonder de invloed de geheugentransfers.

```
    cudaEvent_t start_cuda, stop_cuda;
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);
    cudaEventRecord(start_cuda);

    // Launch kernel (n blocks of N threads)
    vecReverse<<<nrBlocks, nrThreads>>>(d_rVec, d_vec, sizeVec);

    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda);
    cudaEventSynchronize(stop_cuda);
    float ms;
    cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
```

De resultaten die we bij verschillende array sizes en blocks bekomen zijn te zien in tabel 2.

4 Conclusie

Tijdens het testen zagen we dat de uitvoeringstijden voor dezelfde operatie redelijk afwisselend zijn. Een opvallend punt is dat kleinere arrays niet altijd sneller zijn dan grotere arrays. Dit komt waarschijnlijk doordat de overhead van de GPU aanspreken veel groter is dan de daadwerkelijke rekentijd voor deze operatie.

Daarnaast merkten we op dat de allereerste run na het builden vaak veel langer duurt (tot wel 4ms, zie ook outlier), vooral bij grotere wijzigingen van het programma. Dit zou het resultaat kunnen

Array size = Threads	Blocks	Times (ms)	avg time (ms)
10	6	0.523904, 0.710816, 0.538912, 0.427520, 0.900928	0.620416
100	6	0.429088, 0.713120, 0.477056, 0.581472, 0.614496	0.563046
1000	6	0.316128, 0.638976, 0.921568, 0.649216, 0.528512	0.610880
10	1	0.450912, 0.551264, 0.426208, 2.336224, 0.297248	0.812371
100	1	0.483488, 0.315072, 0.407552, 0.501376, 0.313344	0.404166
1000	1	0.358752, 0.422048, 0.370688, 0.269056, 0.344864	0.3530816

Tabel 2: Timing resultaten zonder dataoverdracht

zijn van de GPU die zich bij de eerste keer nog op bepaalde manieren moet initialiseren, waarvoor bij de herhaling van programma's dit reeds is gebeurd en er minder overhead is. En/of dat externe factoren, zoals andere draaiende processen die de GPU gebruiken, invloed hebben op de variatie in de resultaten.

Het verschil tussen de metingen met en zonder `cudaMemcpy` is wel zeer duidelijk. Wanneer we de geheugen transfer meerekenen, stijgt de gemiddelde tijd. Hoe groter de array wordt hoe groter dit verschil, tot wel dubbel zo lang in het geval van een array size van 1000 en 1 block. **We kunnen concluderen dat in het geval van grotere arrays, de data overdragen naar de GPU en terug, meer invloed heeft dan het uitvoeren van deze grotere array, dit kan dus een bottleneck worden.**